

Harness 到底是什么？看看 OpenClaw、Hermes、Claude Code 的演绎吧

原创 叶小钗 叶小钗 2026年4月23日 08:28 四川

AI训练营 9期，5月7日 开班，欢迎咨询

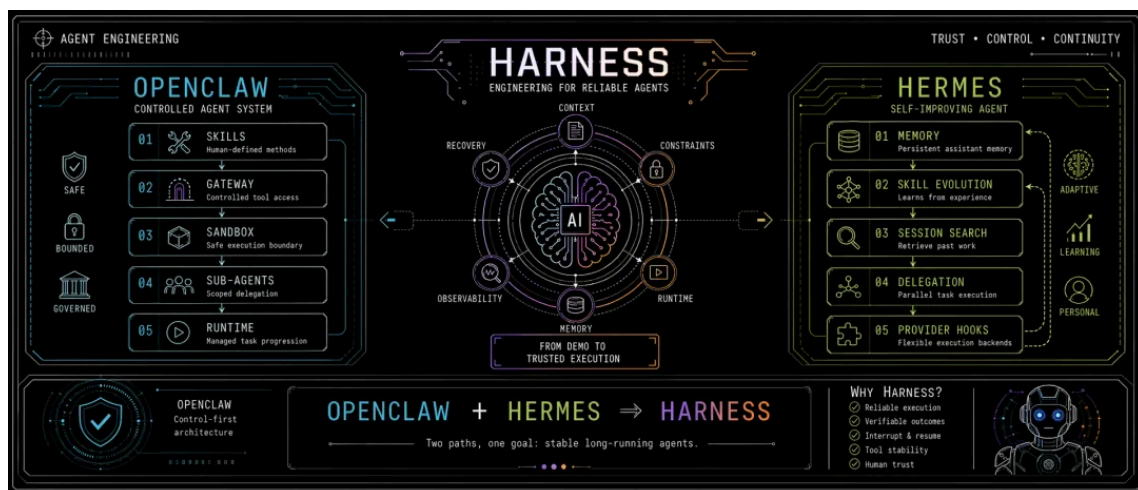
书接上文：

1. [《理论：从工程角度为你拆解 OpenClaw》](#)
2. [《万字：拆解 OpenClaw：从 Gateway、Memory、Skills、多 Agent 到 Runtime》](#)
3. [《万字：拆解 OpenClaw 上下文工程/记忆系统》](#)
4. [《万字：OpenClaw 核心机制 Skills 全解析》](#)
5. [《实践：实现一个Mini-OpenClaw》](#)
6. [《万字：OpenClaw vs Hermes》](#)
7. [《万字：从 Claude Code 看 Harness》](#)
8. [《这可能是 Harness 的诞生过程》](#)

Harness 最近有些小火，但这东西跟 OpenClaw 和 Hermes 不一样，他活得不太真实，到现在都只有个框架性描述：为 Agent 的稳定执行而生。

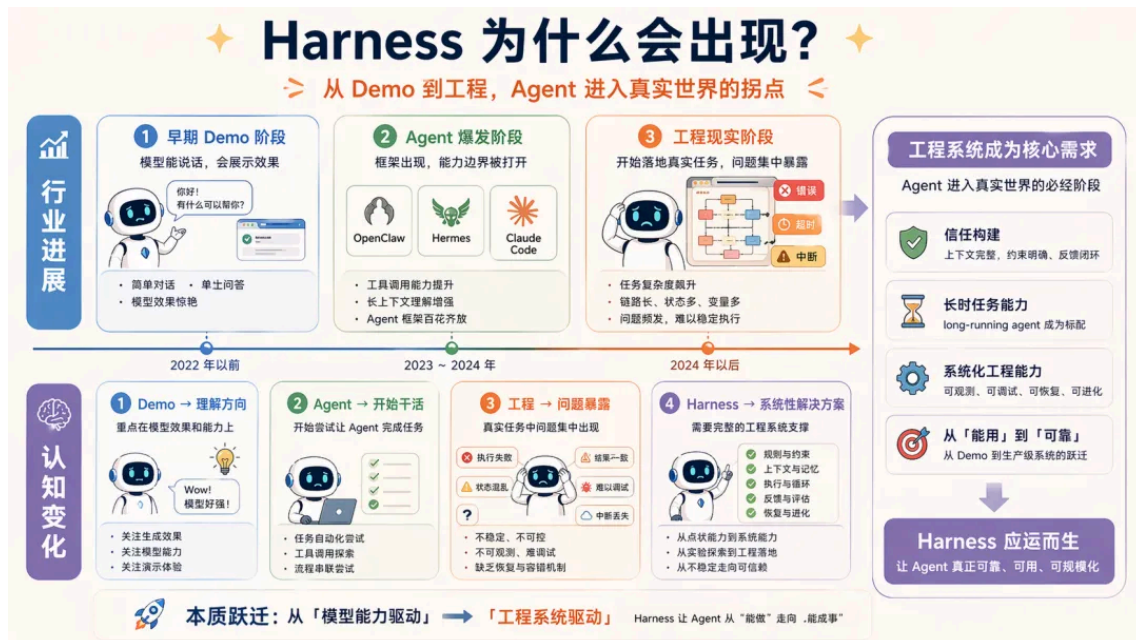
大家在平台上看到的文章，不是太虚、就是太碎。

所以要了解 Harness 不仅要看大概念，最好借助现在实际运行的很好的 Agent 框架，比如 Claude Code、OpenClaw、Hermes，这样才能把它重新拉回工程现场。



Harness 诞生背景

得益于近来 Agent 的发展，包括 OpenClaw、Hermes 的相继发布与 Claude Code 的源码泄露，全世界对 Agent 开发范式的理解都进入了一个新的台阶；



在这个基础下，我们不会认为 Harness 这个词是突然火起来的，**因为 Agent 真开始干活以后，工程问题终于藏不住了。**

还是那句话

Demo 可以让我们明白范式的大方向，工程才能真正让我们知道范式是什么

Martin Fowler 在 2026 年 4 月写的文章里，把 Harness Engineering 直接定义成一套围绕 coding agent 的信任建设模型，核心是通过上下文、约束、反馈回路和工程结构，让人逐步敢把任务交给 Agent。

Anthropic 自己也在官方工程文章里直接把 Claude Code 叫作一种优秀的 harness，并且进一步讨论了 long-running agents 和 long-running application development 里的 harness design。

PS：所以现阶段 Claude 老是喜欢强调自己强的不只是模型还有工程，但其实我们用国内的框架，只要换了 Claude 模型后能力也会上升不少

所以，我觉得 Claude 强的还是 Coding 这个点，国内的工程能力未必比他弱

所以，我们今天探讨工程范式的集大成者 Harness，自然不能还停留在提示词工程打转，上下文工程似乎也不足以包含他的含义，现阶段问题已经回归到了：

为什么 OpenClaw、Hermes、Claude Code 这些 Agent 框架，最后都会长出一整套工程系统？

而这套系统，为什么越来越像 Agent 成败的关键？

模型与工程

过去两年，大模型公司主要在围绕 Agent 生态卷：

1. 语义理解
2. 视觉生成
3. 长上下文
4. 工具调用
5. 多模态
6. 电脑操作、浏览器操作之类的 Agent 能力

业界也有一个思路：面向未来半年做设计，因为那时候模型更强了，工程侧的成本就会更低，这里就是一个重要假设，只要模型不断更强，应用自然就会自己长出来。

但实际情况是长上下文和 tool calling 的稳定性上来以后，Agent 这条线确实一下子变得好做了很多。

但问题是：模型强，不等于工程就稳。总有很多跳出的边界，包括：

1. 模型无论如何依旧会工具调用不准、不稳；
2. 模型能理解复杂输入，但在持续推进一个长任务时候依旧吃力；
3. 模型能写出代码，不代表它知道自己到底写对了没有；
4. ...



而工程架构的意义在于，让 Agent 稳定地把事情做完，也正因为如此，2025 年到 2026 年，Agent 讨论的重心开始明显变化了：

1. 以前大家讨论 Prompt 怎么写
2. 后来讨论 Context 怎么喂
3. 现在真正开始讨论：Agent 运行起来以后，还缺什么系统能力

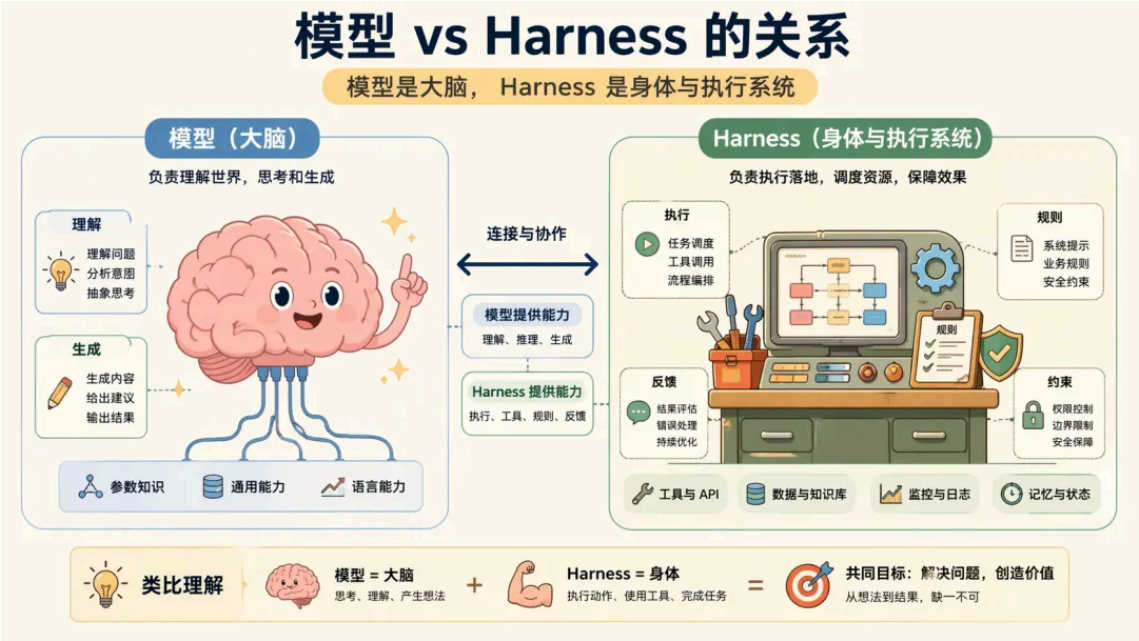
以上就是 Harness 出现的整个场景了。

什么是 Harness

现阶段市面上有很多对 Harness 的定义，比较容易理解的是：

模型 = 大脑

Harness = 身体 + 工作台 + 操作规程 + 监督机制

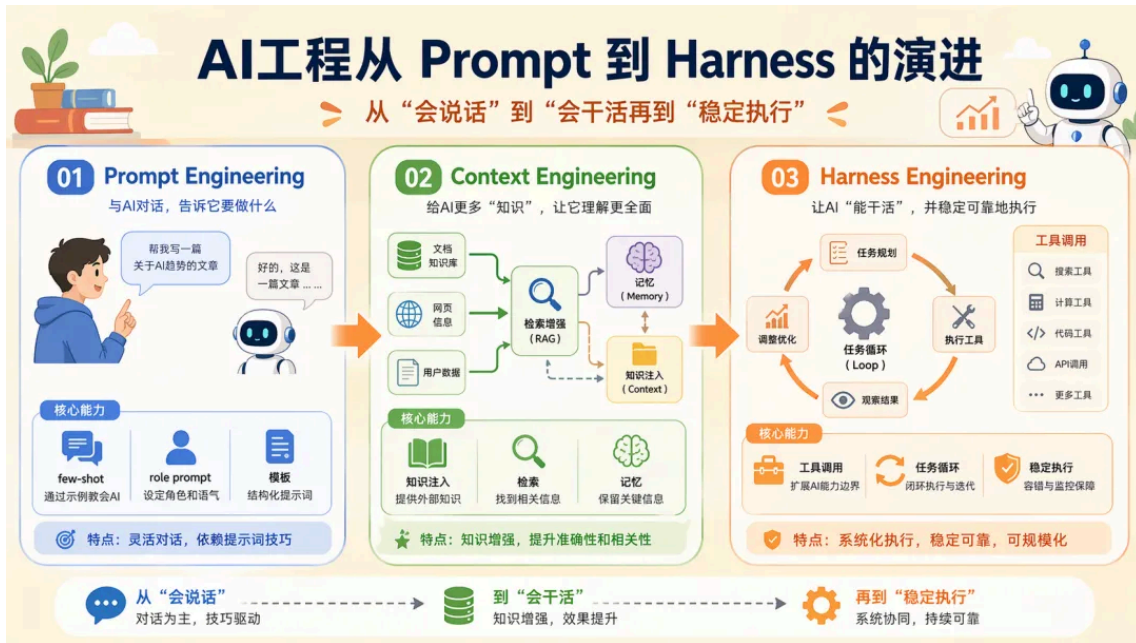


首先不说是否严谨，这种描述，我其实有点不知道如何做展开，因为 Harness 是工程产物，工程产物就一定不是一个 SDK、或者提示词小技巧，他是我们在项目中啃的各种硬骨头的集合，所以：

Harness，是把模型能力变成持续、稳定、可验证产品能力的那套系统集合

说到底，就是很多规则约束和设计了...

Prompt → Context → Harness



我们前面说了，Harness 是我们在做 Agent 过程中工程实践的产物，所以 Harness 不是凭空冒出来的，在之前的工程产物是：Prompt Engineering 和 Context Engineering，所以：

上下文工程是提示词工程的延续，Harness 是他们两种延续的结果

1. Prompt Engineering

提示词工程直接解决的是我们应该如何与模型交互，所以他是 最简单而有效的，所以最早大家关注的是：

1. few-shot
2. role prompt
3. CoT
4. 输出格式约束
5. 提示词模板
6. ...

这一层的本质，是把行业 know-how 翻译成自然语言指令。

这里值得进一步说明的是：无论工程怎么演进，最终都会回到提示词，所以很多人认为现阶段的各种工程优化，依旧是提示词工程的延伸，这种认识不能说错。

2. Context Engineering

后面任务复杂了，光写一句好 prompt 已经不够了，于是开始出现上下文工程：

1. 哪些私有知识带进来
2. 哪些历史聊天保留

3. 如何压缩超长上下文
4. 如何做检索
5. 如何让模型不失忆，也不被信息淹没
6.

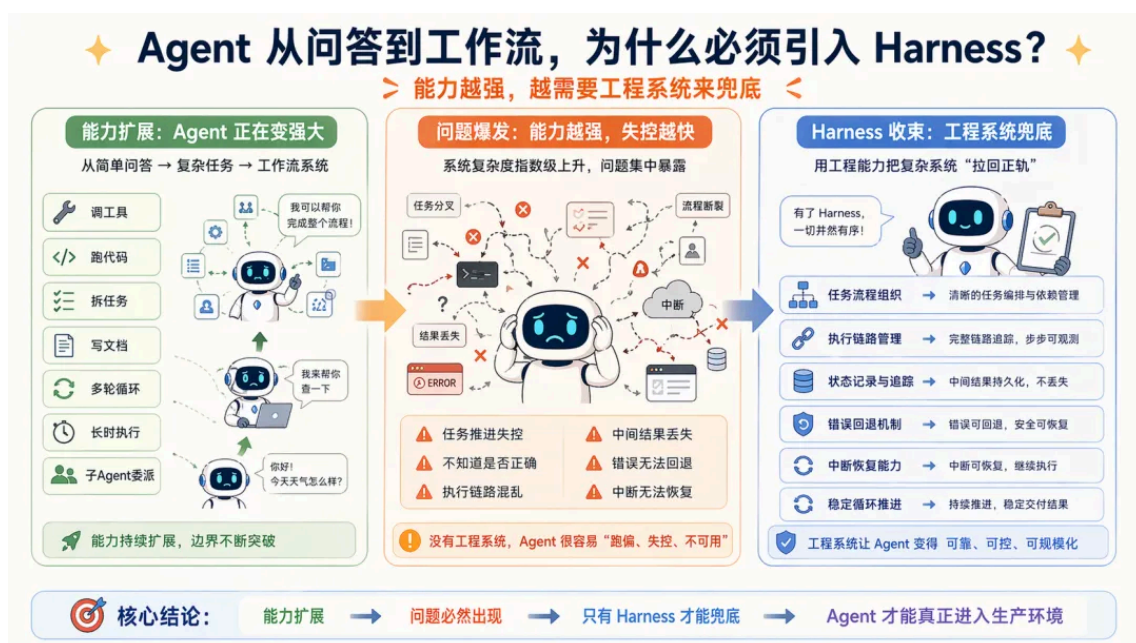
到了这一步，系统已经不是单纯会照着SOP作答，而是开始会结合材料答，这里的核心都是围绕 CoT展开的。

应该说 上下文工程 的核心是 数据工程，然后真正做生产级 AI 应用的人会去到一个怪圈：80% 的时间都在搞数据，我们甚至会怀疑这种枯燥而烦躁的工作跟炫酷的 AI 有什么关系？

3. Harness Engineering

再往后，Agent 开始不满足于问答了，它开始：

1. 调工具
2. 跑代码
3. 拆任务
4. 看页面
5. 写文档
6. 多轮循环
7. 长时执行
8. 子 Agent 委派
9. 中断恢复
10. 测试与验收
11. ...



我们前面说过，Agent 的出现是要解决 Workflow 泛化能力不足而导致的繁重维护工作。

而因为工程复杂度加剧，这时上下文工程也不够了。因为问题已经从数据范畴变成了：

1. 任务怎么持续推进而不失控
2. 模型怎么知道自己做对了没有
3. 执行链路怎么组织
4. 中间结果怎么留痕
5. 错了怎么回退
6. 停了怎么续上
7. ...

这时候，Harness 就自然冒出来了，他很合理：

当 Agent 从问答走向工作流，从单轮走向长链任务时，被工程现实逼出来的一套总解决方案

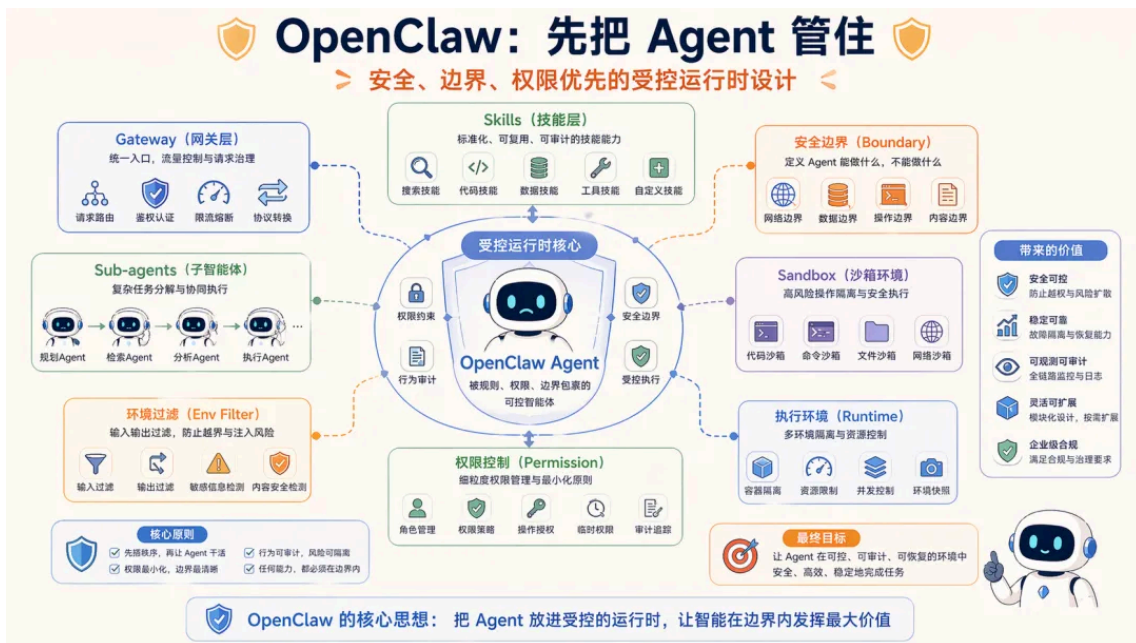
OpenClaw、Hermes

前面我们说过，现在 Harness 被搞得很空，因为我们总喜欢脱离真实框架。

所以，真的要聊就一定要回归 Agent 本身，把 Harness 放回 OpenClaw、Hermes、Claude Code 里，它一下子就具体了。

因为这三个东西，分别代表了三种很典型的 Agent 工程取向：

1. OpenClaw：先把 Agent 管住



OpenClaw 的官方文档和仓库公开能力，很明显是偏受控运行时的。

它把 Skills、Gateway、安全边界、Sub-agents、Sandbox 都拆得很清楚。

比如官方 Skills 文档就写得很明确：OpenClaw 使用 AgentSkills-compatible 的 skill folder，每个技能目录里有 SKILL.md，并且在加载时会按环境、配置和依赖做过滤。

它的安全文档也反复强调：OpenClaw 当前假设的是 personal assistant security model，也就是一个信任边界内的个人助理部署，而不是无边界生产放权。

这种设计背后的 系统工程目标 很清楚：先把权限、边界、角色、技能、执行环境组织起来，再让 Agent 干活。

因为 OpenClaw 有个目标，还是想让他成为真正的企业 Agent 标配，所以他的工程方向也很明确：

怎么让 Agent 安全、稳定、受控地执行任务？

只不过，现阶段这东西确实不成熟，尤其在多人协作这块很难，所以他没做好，但不能说他方向错。

2. Hermes：先让 Agent 长本事

Hermes 的 README 则是另一种味道。



它自己把自己定义为 “the self-improving AI agent”，而且直接把核心能力写成一串学习闭环：

1. creates skills from experience
2. improves them during use
3. nudges itself to persist knowledge
4. searches its own past conversations
5. builds a deepening model of who you are across sessions

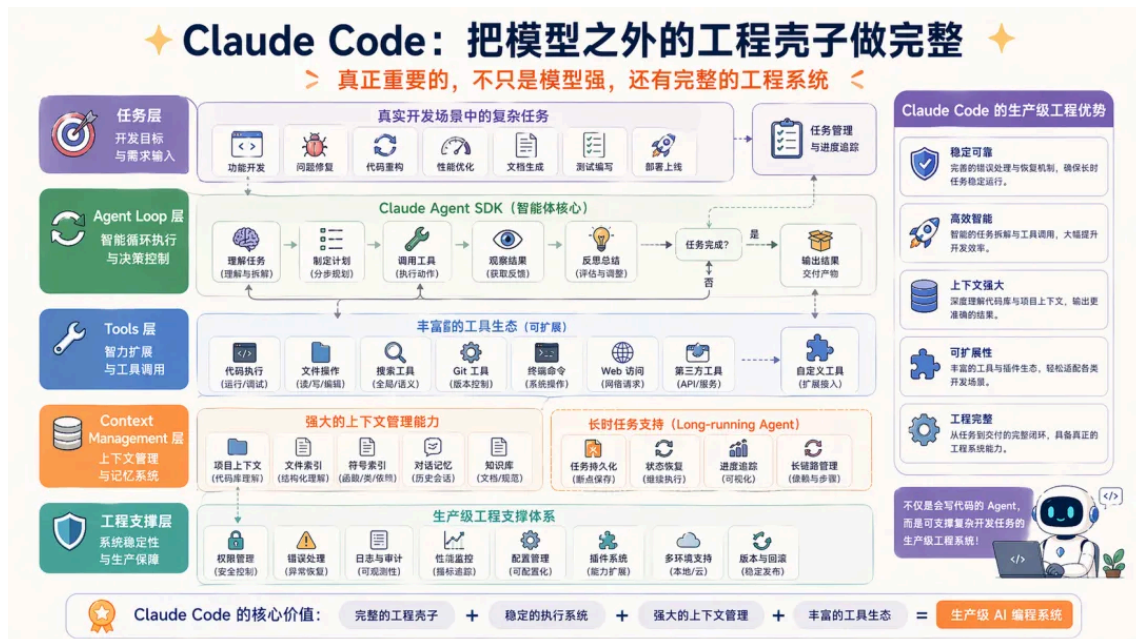
Hermes 的官方文档还明确提供了 8 种 external memory provider，并说明 built-in MEMORY.md / USER.md 始终存在，同时只能启用一个外部 provider，以避免 schema 膨胀和冲突。

这也是我为什么喜欢说 Hermes 鸡贼的原因：他没有 OpenClaw 那种野心，他暂时更多只想让每个人用好，甚至就是指着 OpenClaw 痛点在做迭代，所以其工程目标也很清晰：

先让 Agent 学会从经验中成长，再慢慢补边界和治理

Hermes 的目标是：怎么让 Agent 越用越强、越用越像一个长期助手？

3. Claude Code



Claude Code 面临的场景就完全不同了，他是正儿八经生产级别应用，它已经不是单纯“一个会 coding 的 agent”了。

Anthropic 官方现在已经把与 Claude Code 同源的能力开放成 Claude Agent SDK，明确说这套 SDK 提供的正是 Claude Code 背后的 tools、agent loop 和 context management。

同时，Anthropic 又连续写了几篇工程文章，专门讲：

1. 长时 agent 的 harness 怎么设计
2. application development 场景下 harness 怎么优化
3. Claude Code 为什么本身就是一个优秀 harness

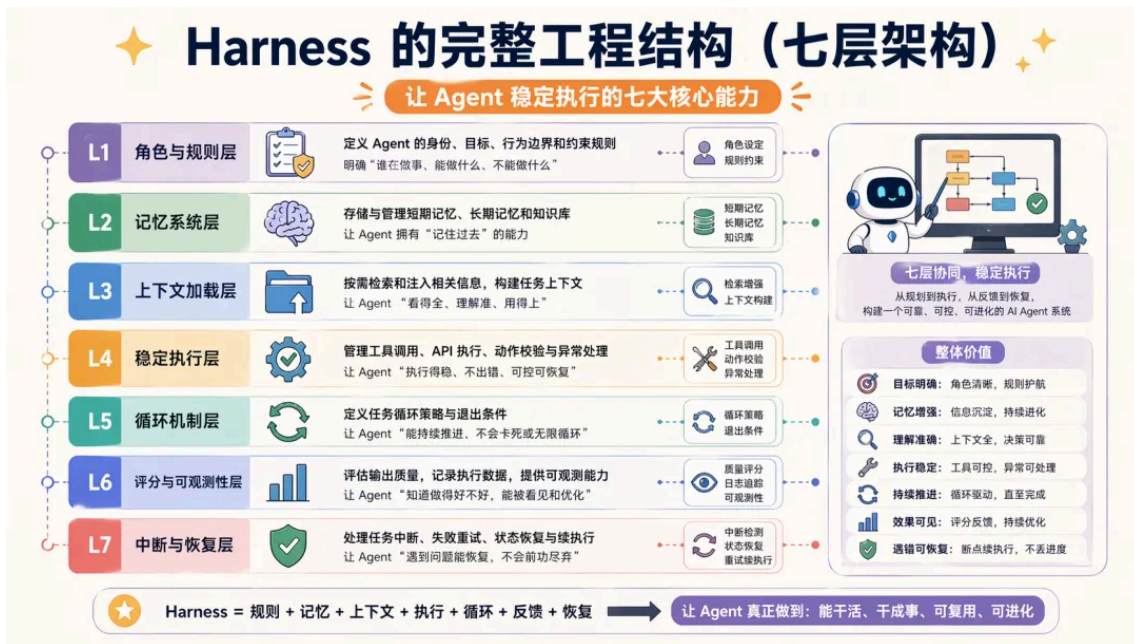
也就是说，Claude Code 的价值不只是模型强，而是：

它已经把模型之外那一整套工程壳子做到相当重要了

所以，现在真的要去学 Harness，逻辑上 Claude Code 是最好的典范，但他的完整代码我们看不到，而且从复杂度来说，OpenClaw 应该是最优解。

拆解 Harness

如果真要把 Harness 拆开看，我认为至少有七层



这里除了我自己之前 AI 应用的认知外，而是每一层都尽量用 OpenClaw、Hermes、Claude Code 来落地。

第一层：角色与规则

一个模型接到任务后，第一件事其实不是调工具，而是先明确：

1. 它是谁
2. 它负责规划、执行还是验收
3. 它边界在哪
4. 它碰到不确定情况怎么办
5. ...

只要这个顶住了，后面所有动作就会有基本的控制力。

OpenClaw 在这件事上做得很标准：Skill 是人写的，规则是人定的，边界是系统设的，Agent 更多是在框架内执行。

Hermes 在这件事上做得很灵活：它也有系统提示词、角色定义和运行时规则，但它更愿意把一部分能力判断交给 Agent 自己，意思是你别去管什么 Skills 广场了，比如什么时候生成新 Skill，什么时候更新旧 Skill。

Claude Code 这边则更接近 **工具即流程**：Anthropic 不断强调 agent loop、context management、长时任务 initializer / coding agent 分工，这其实就是把角色和节奏预埋进系统。

所以做 Harness 最先确定的是你现在以什么身份工作。

第二层：记忆系统

任务一旦变长，就一定会产生很多中间结果：

1. 拆出来的子任务
2. 讨论过的方案
3. 当前做到哪一步
4. 用户偏好
5. 历史错误
6. 成功经验
7. ...

再长的上下文都不够糟蹋，于是不同框架的工程差异也出现了：

OpenClaw 对记忆的态度很克制，本质上更接近 **可替换能力位**，意思是我实现最基础的，你按照自己的情况做实现来替换我；

Hermes 则把记忆做成完整体系：内置 MEMORY.md、USER.md，叠加 external memory provider，再叠加 session search；

官方文档明确说 built-in memory 始终启用，外部 provider 只允许同时存在一个，还是那个原则：**你别折腾了，就用我这个吧？**

所以现在用户有个感受：**OpenClaw 经常不理我昨天说了什么**，而虽然 Hermes 也有这个情况，但我会知道为什么...

Claude Code 这边则在官方文章里更强调另一条思路：**长时任务里，清晰 artifact 和 handoff 特别重要，让下一次 session 能接着做。**

所以在系统工程里，记忆系统的本质一直都是在围绕，**任务过程能不能留下痕迹，系统下次还能不能接上做展开。**

第三层：上下文加载机制

到底给模型看什么？ 这是所有 AI 应用都会遭遇的难点，这个甚至有怎么做都不是最优解的感觉...

因为真实 Agent 场景里，模型前面能看的东西会越来越多：角色与规则、历史对话、记忆、技能、工具结果、当前任务.....

这里问题就来了：**不是信息不够，而是信息太多。**

OpenClaw 的 Skills 加载逻辑，本质就是一种上下文过滤：**按环境、配置和依赖去筛。**

Hermes 则做了另一条路：它的 session search 不是把历史原文一股脑塞回来，而是先检索，再经过处理；它还支持 context engine plugin，用插件来替换内置上下文压缩器。

所以 **如何在每一轮只给模型当前最需要的那部分**，这个在我看来是所有模型工程最难的点，因为这

里进一步衍生就会涉及到私有数据加载的问题了，做不好这层，系统就会两头出问题：

1. 看得太少，像失忆
2. 看得太多，开始变蠢

第四层：稳定执行

Agent 或者说是 ReAct 框架，是在模型时代我们选定的框架，从这块开始 Agent 就开始动手了，所以：

1. 工具怎么接
2. 命令怎么跑
3. 文件怎么读写
4. 页面怎么查看
5. 代码怎么执行
6. 结果怎么回收
7.

这些 Tools 动作全部是工程需要关注的，因为他们依赖于第三方，必定经常出问题啊！

OpenClaw 在这块是典型 **安全优先** 的运行时。

Hermes 则更像 **执行后端可切换**，官方 README 写明它可以跑在本地、VPS、GPU 集群和接近零空闲成本的 serverless 环境。

所以 Harness 的这一层是：把语言判断，稳定地变成真实动作。没有这一层、这一层做不好，就会经常出错。

第五层：有效循环

普通聊天已经是 AI1.0 的产物了，从 DeepSeek 以后，我们就在追求多轮问答，而 Agent 会因为要处理复杂的问题，不可避免的进入循环：

1. 理解任务
2. 决定下一步
3. 执行
4. 读结果
5. 判断下一步
6. 一直循环到收口

OpenClaw 的多 Agent、skills、runtime 其实也都在围绕循环推进做。

Hermes 则把 delegate、skills、memory、search、provider hooks 都嵌在这个循环里。

我们前面说过：更多的智能必定会消耗更多的 Token，其实 Agent 循环的问题也就在这里：会不会空耗 token 和时间，却没有实质推进。

在工程系统里面，担心的一直不是循环，而是钱花了，事情没干...

第六层：评分与可观测性

模型最大的问题之一，不是不会做，而是经常觉得自己已经做完了。

表面上看，代码写出来了，页面也渲染了，回复也发出去了，好像事情已经闭环了，但只要一验，就会发现很多地方根本没通。

所以我们在系统工程中对每个重要节点都会埋点，为的就是建立评分与可观测性机制。

也就是说，系统不能只听模型自己汇报“我完成了”，而是要能通过测试、日志、页面验收、运行指标、人工审查、Benchmark 等方式，真实地看到它做了什么、做到什么程度、结果到底好不好。

如何让我们对 agent 结果建立信任，这个信任不可能只靠模型自述建立，它必须有外部反馈机制。

Anthropic 的 harness design 文章也在讲类似问题：要让长时 application development 表现更好，光有 agent loop 不够，还得有更强的环境和反馈壳子。

OpenClaw 在这里的策略是制度化：靠规则、沙箱、受控执行去约束结果。

Hermes 在这块则更偏学习闭环：把执行结果、错误路径、成功经验逐步沉淀成 Skill 或 Memory。

所以这一层的目标就是：不要让模型稀里糊涂自己给自己打高分。

中断修复

这层是工程控制的关键。

因为我们习惯性的是一次性做好，但真实世界并不是那么回事，并且我们人类在设计 SOP/Workflow 的时候也相当不适应边界回退，那么模型这里也会面临类似的问题。

所以，模型在循环往复的时候，整体 SOP 会不会后退、如何后退就很关键了...

这一层平时看起来就烦，但真跑起来以后特别重要，因为你会发现你的任务真的会中断、会超时、会切 session、会失败重试.....

至于如何解决：

Hermes 这里是通过 MEMORY、USER、session search、external provider 把接续这件事做成系统能力。

OpenClaw 的思路则更偏流程与痕迹受控。

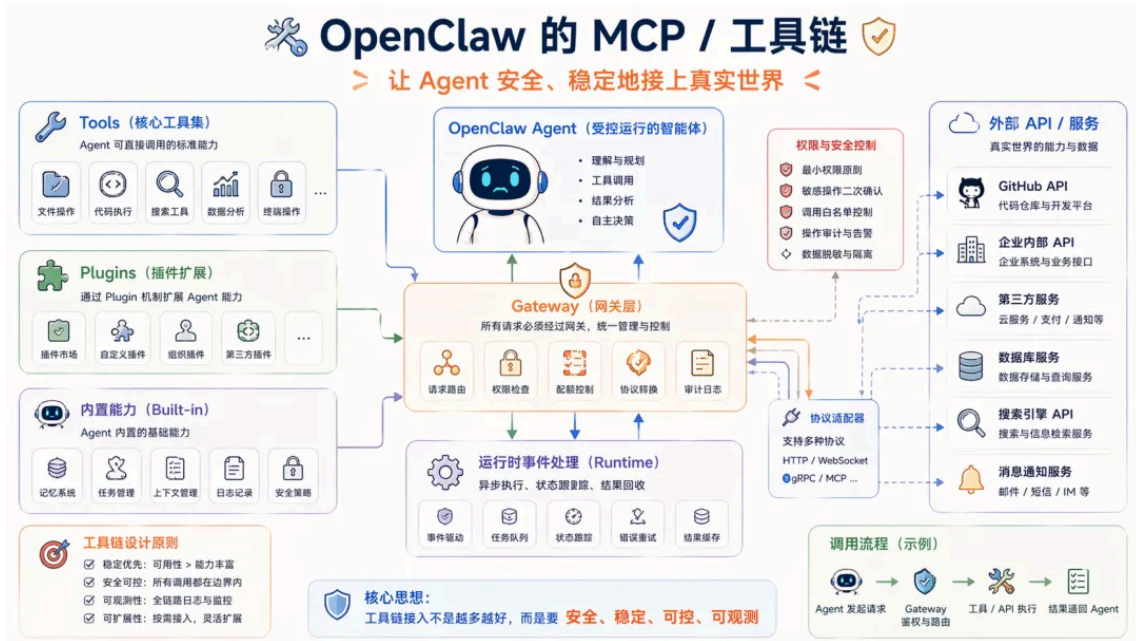
所以我们做工程系统，最后面对这一层：如何把把断掉的任务重新接起来？

到这里，我认为基本聊清楚了，最后在更具体的用 OpenClaw 来带着大家过一次 Harness。

OpenClaw：理解 Harness

刚刚我们用概念聊了很多关于 Harness 的内容，那么 当一个 Agent 框架真的跑起来以后，这套所谓的 Harness，具体到底长什么样？这里继续以我较为熟悉的 OpenClaw 做展开。

第一，MCP/工具链



很多人一提 OpenClaw，第一反应是 Skills，这没错，Skills 确实很核心，并且也是每个人跟 Agent 交互的入口。

但如果从 Harness 工程稳定性 的角度看 MCP/工具链这一整层会很重要，因为一个 Agent 真开始工作以后，需要解决到底怎么安全、稳定地接上真实世界。

Skills 是方法稳定器，让模型不要过于发散；MCP/工具链就是能力本身

Skills 出问题，系统就会乱做，同样的输入不同的输出；但 Tools 出点问题，整个流程就断了，而且这是依赖于第三方的，本来就容易出问题，包括：

1. API 变动；
2. 权限变动；
3. 插件失效、插件参数变化；
4. ...

所以这里工程系统会先定义清楚能力规范，OpenClaw 这点很典型，把 Tools、Plugins、Gateway、外部能力接入都放进一个明显有边界感的系统里，其目的是：

模型能不能在一个被约束的能力平面里稳定地调工具

比如最常见的一种情况：某个外部 API 挂了。

如果没有工程控制，模型可能根本分不清这是自己理解错了，还是上游接口挂了；

在这个场景下，模型会表现得像个蠢货，加大力量输出不停循环，白白空耗 token 和时间，比较夸张的就是直接告诉下游：我搞定的...

这时候，Harness 的价值就出来了。

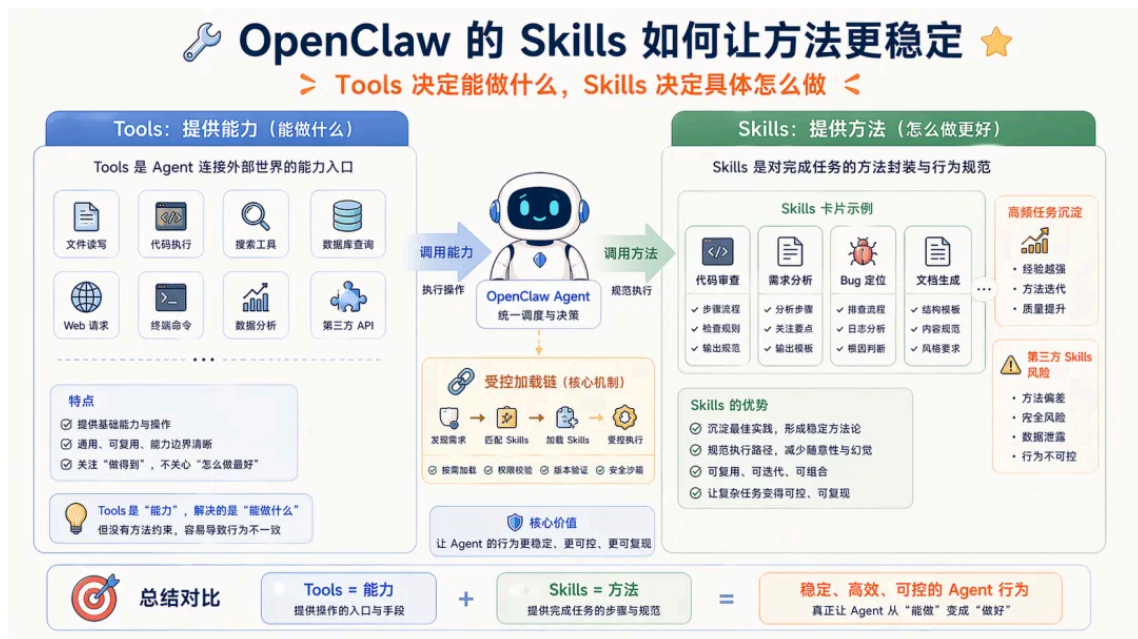
以 OpenClaw 这类系统为例，正确的处理思路是把 API 调用失败当成一个运行时事件来处理。

OpenClaw 现阶段的策略，是将工具调用进入了一个受 Gateway 管理的运行时平面。具体这里细节太多，我们不展开...

第二，Skills

在能力底座搭起来以后，就轮到 Skills，这东西非常重要

1. Tools 解决能做什么；
2. Skills 解决这些事具体怎么做；



Skill 这种东西天然很好，从诞生时候的按需加载就可以部分缓解 Tools 调用错误的问题，其 Workflow 提示词会进一步带来稳定性，比如：可以把高频任务的方法沉淀下来。

但 OpenClaw 这种平台型 Agent 中 Skills 问题也同样明显：

1. skill 可能来自第三方
2. skill 本质上会进入 prompt 构造链路
3. 模型本来就脆弱，很容易被恶意或低质量提示词污染
4. 一旦 skill 机制失控，Agent 的方法层就会整体失真

所以在系统工程中，在 Skills 这套机制但是前，他本身就应该被归类到 Harness，我们之前就有类似的实现；

而在如今 Skills 被底层实现后，我们就不太关心 Skills 的意义了，比如 OpenClaw 关心的是：

怎么让 Skills 这套开放机制，不至于把整个系统拖垮

这里又是用各种规则做约束，大家会发现工程系统会产生非常多的约束，比如 OpenClaw 这里的先强调，**第三方 Skills 本来就是不可信的**

其次，OpenClaw 也还需要做进一步的兜底策略，这里的办法是把 Skills 放进受控加载链里，比如：

1. plugin skills 只是低优先级路径，同名 skill 会被 bundled / managed / agent / workspace skill 覆盖；
2. workspace 和 extra-dir 的 skill discovery，只接受解析后 realpath 仍留在配置根目录内的 skill root 和 SKILL.md，避免路径穿越和任意逃逸；

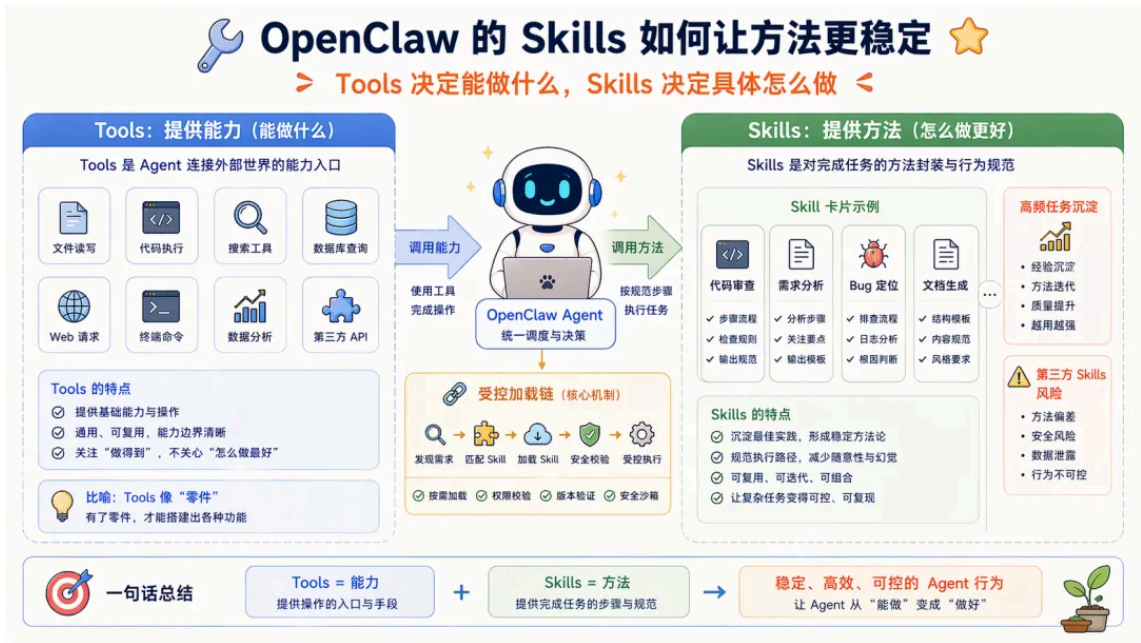
然后这里还有很多策略做兜底，细节展开太多，这里也不继续了...

第三，Runtime

再往下的问题就不是工具与技能调用的问题了，而是一件复杂任务，到底怎么持续推进。

OpenClaw 在执行复杂任务时会进入一个循环：

1. 先理解问题；
2. 再决定下一步；
3. 然后调工具、读文件、跑代码；
4. 再看返回结果；
5. 再判断接下来该做什么；
6. 一直循环到任务真正收口；



只不过真实情况都是 BUG 频出, 模型一旦进入长任务, 就会开始出现各种问题:

1. 它可能跑着跑着提前收尾, 明明事情还没做完, 就告诉你已经处理好了;
2. 它可能做了一半又绕回原地, 重复调用同一个工具;
3. ...

所以, 从工程侧一点会希望有个东西去搞清楚:

1. 当前任务到哪一步了
2. 下一步应该谁干
3. 什么时候继续
4. 什么时候暂停
5. 什么时候打回
6. ...

而 OpenClaw 的 Runtime 就是在承担这件事, 他会尝试把 Agent 的行为从一堆零散动作, 组织成一条真正能推进任务的流程。

这个 RunTime 会包括整个项目的可观测性和中断重试的逻辑, 有点复杂, 这里依旧不展开...

但是这里大家也许对 Harness 是什么有了更进一步理解了。

结语

Harness 不是一个模块, 而是一条路, 一条咬硬骨头走出来的方法论。

大家可以看清楚 一个 Demo Agent 是如何一步步走到 OpenClaw 的:

1. 一开始只是接工具;
2. 然后发现工具不稳, 要加规则;
3. 再发现规则不够, 要加 Skills;

- 4. 再发现 Skills 还不够，要加 Runtime 和 Workflow；
- 5. 再发现任务会假完成，就要补评分与可观测性；
- 6. 再发现任务会中断，就得补恢复能力；
- 7. ...

当所有的问题都被覆盖得差不多的时候，发现所有的动作，已经不能用一个小优化来做说明了：

Agent 从会答，到会做，再到能稳定做完，整条链上缺的所有工程能力，这就是 Harness 了...

所以 Harness 以后未必还叫 Harness,但这条路，肯定不会消失。



叶小钊

鹅厂、ctrip、baidu、一线开发，B站技术专家，独角兽技术负责人，AI产品项目负责...
204篇原创内容

公众号

点击上方卡片关注叶小钊公众号，查看下方二维码，添加我个人微信：

叶小钊

3年AI实践 + 2次AI创业
大型AI项目（预算过亿）负责人

医疗AI

英语AI

项目管理AI

AI客服

GEO

其他

添加微信好友



每天一点干货
加群共同学习进步

往期推荐

《系统性：如何进入AI行业？》

《万字：Agent概述》

《万字：做一个Agent-上》

《万字：做一个Agent-下》

[📖 《万字：理解LangChain》](#)

[📖 《实践：用LangChain开发一个Agent》](#)

[📖 《万字：AI Coding 的真实情况》](#)

[📖 《万字：普通人学 AI Coding》](#)

[📖 《AI Coding 实战：10年祖传系统，54万行代码，2周重构结束》](#)

[📖 《AI Coding 实战：2周重构54万行代码，细节详解》](#)

[📖 《重要：AI学习路线图》](#)

[📖 《万字：个人IP，包教包会》](#)

[📖 《万字：AI客服实战方法论》](#)

[📖 《万字：生产级别的RAG系统》](#)

[📖 《万字：RAG实战技巧，包教包会》](#)

[📖 《2025年终总结》](#)

[📖 《OpenClaw 会不会淘汰 Coze、Dify 这类平台？》](#)

[📖 《别被 OpenClaw 带偏了，AI 公司到底该如何组织人才？》](#)

 喜欢作者